

Spring Testing Exercises

Exercise 1: Basic Unit Test for a Service Method

CalculatorService.java :

```
import org.springframework.stereotype.Service;

@Service
public class CalculatorService {

    public int add(int a, int b) {
        return a + b;
    }
}
```

CalculatorServiceTest.java :

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class CalculatorServiceTest {

    @Test
    void testAdd() {

        CalculatorService service =
            new CalculatorService();

        int result = service.add(2, 3);

        assertEquals(5, result);
    }
}
```

Output :

Tests run: 1
Failures: 0

Exercise 2: Mocking a Repository in a Service Test

User.java :

```
import jakarta.persistence.Entity;
import jakarta.persistence.Id;

@Entity
public class User {

    @Id
    private Long id;
    private String name;

    public Long getId() {
        return id;
    }
}
```

```

    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}

```

UserRepository.java :

```
import org.springframework.data.jpa.repository.JpaRepository;
```

```
public interface UserRepository
    extends JpaRepository<User, Long> {
}

```

UserService.java

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
```

```
@Service
public class UserService {

    @Autowired
    private UserRepository userRepository;

    public User getUserById(Long id) {
        return userRepository.findById(id)
            .orElse(null);
    }
}

```

UserServiceTest.java :

```
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
```

```
import java.util.Optional;
```

```
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;
```

```
@ExtendWith(MockitoExtension.class)
public class UserServiceTest {
```

```
    @Mock
    private UserRepository userRepository;
```

```
    @InjectMocks
```

```

private UserService userService;

@Test
void testGetUserById() {

    User user = new User();
    user.setId(1L);
    user.setName("John");

    when(userRepository.findById(1L))
        .thenReturn(Optional.of(user));

    User result = userService.getUserById(1L);

    assertNotNull(result);
    assertEquals("John", result.getName());
}
}

```

Output :

Tests run: 1
Failures: 0

Exercise 3: Testing a REST Controller with MockMvc

UserControllerTest.java :

```

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.test.web.servlet.MockMvc;

import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@WebMvcTest(UserController.class)
public class UserControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private UserService userService;

    @Test
    void testGetUser() throws Exception {

        User user = new User();
        user.setId(1L);
        user.setName("John");

        when(userService.getUserById(1L))
            .thenReturn(user);

        mockMvc.perform(get("/users/1"))
    }
}

```

```

        .andExpect(status().isOk())
        .andExpect(jsonPath("$.id").value(1))
        .andExpect(jsonPath("$.name").value("John"));
    }
}

```

Output :

Tests run: 1
Failures: 0

Exercise 4: Integration Test with Spring Boot

UserControllerIntegrationTest.java :

```

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.test.web.client.TestRestTemplate;

import static org.junit.jupiter.api.Assertions.*;

@SpringBootTest(webEnvironment =
    SpringBootTest.WebEnvironment.RANDOM_PORT)
public class UserControllerIntegrationTest {

    @Autowired
    private TestRestTemplate restTemplate;

    @Test
    void testGetUser() {

        User user =
            restTemplate.getForObject(
                "/users/1",
                User.class
            );

        assertNotNull(user);
    }
}

```

Output :

Tests run: 1
Failures: 0

Exercise 5: Test Controller POST Endpoint

UserControllerTest.java :

```

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;

```

```

import org.springframework.test.web.servlet.MockMvc;

import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@WebMvcTest(UserController.class)
public class UserControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private UserService userService;

    @Test
    void testCreateUser() throws Exception {

        User user = new User();
        user.setId(1L);
        user.setName("John");

        when(userService.saveUser(org.mockito.ArgumentMatchers.any(User.class)))
            .thenReturn(user);

        String json =
            "{\"id\":1,\"name\":\"John\"}";

        mockMvc.perform(post("/users")
            .contentType("application/json")
            .content(json)
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.id").value(1))
            .andExpect(jsonPath("$.name").value("John")));
    }
}

```

Output :

Tests run: 1
Failures: 0

Exercise 6: Test Service Exception Handling

UserServiceTest.java :

```

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)

```

```

public class UserServiceTest {

    @Mock
    private UserRepository userRepository;

    @InjectMocks
    private UserService userService;

    @Test
    void testUserNotFound() {

        when(userRepository.findById(1L))
            .thenReturn(Optional.empty());

        User result =
            userService.getUserById(1L);

        assertNull(result);
    }
}

```

Output :

Tests run: 1
 Failures: 0

Alternative (If Service Throws Exception) :

```

@Test
void testUserNotFoundException() {

    when(userRepository.findById(1L))
        .thenReturn(Optional.empty());

    assertThrows(
        RuntimeException.class,
        () -> userService.getUserById(1L)
    );
}

```

Exercise 7: Test Custom Repository Query

UserRepositoryTest.java :

```

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;

import java.util.List;

import static org.junit.jupiter.api.Assertions.*;

@DataJpaTest
public class UserRepositoryTest {

    @Autowired
    private UserRepository userRepository;

```

```

@Test
void testFindByName() {

    User user = new User();
    user.setName("John");

    userRepository.save(user);

    List<User> users =
        userRepository.findByName("John");

    assertEquals(1, users.size());
    assertEquals(
        "John",
        users.get(0).getName()
    );
}
}

```

Output :

Tests run: 1
Failures: 0

Exercise 8: Test Controller Exception Handling

GlobalExceptionHandler.java :

```

import java.util.NoSuchElementException;
import org.springframework.http.*;
import org.springframework.web.bind.annotation.*;

@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(NoSuchElementException.class)
    public ResponseEntity<String> handleNotFound(
        NoSuchElementException ex) {

        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body("User not found");
    }
}

```

UserControllerTest.java :

```

import java.util.NoSuchElementException;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.context.annotation.Import;
import org.springframework.test.web.servlet.MockMvc;

import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;

```

```

import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@WebMvcTest(UserController.class)
@Import(GlobalExceptionHandler.class)
public class UserControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private UserService userService;

    @Test
    void testHandleNotFoundException() throws Exception {

        when(userService.getUserById(1L))
            .thenReturn(
                new NoSuchElementException()
            );

        mockMvc.perform(get("/users/1"))
            .andExpect(
                status().isNotFound()
            )
            .andExpect(
                content().string(
                    "User not found"
                )
            );
    }
}

```

Output :

Tests run: 1
Failures: 0

Exercise 9: Parameterized Test with JUnit

CalculatorService.java :

```

public class CalculatorService {

    public int add(int a, int b) {
        return a + b;
    }
}

```

CalculatorServiceTest.java :

```

import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;

import static org.junit.jupiter.api.Assertions.*;

public class CalculatorServiceTest {

    CalculatorService service =

```



```
        new CalculatorService();

    @ParameterizedTest
    @CsvSource({
        "2,3,5",
        "5,5,10",
        "10,20,30"
    })
    void testAdd(int a,
                int b,
                int expected) {

        assertEquals(
            expected,
            service.add(a, b)
        );
    }
}
```

Output :

Tests run: 3
Failures: 0